1 Part 1

Binary Density Estimation

Modeling $\Pr(x=1)$ given IID samples $x^1, x^2, \ldots, x^n$.

Bernoulli Parameterization

Bernoulli Distribution over binary variable, parameterized by θ.

$$\Pr(x \mid \theta) = \theta^x (1-\theta)^{1-x}$$

Binary Inference Tasks

1. Computing Probabilities
2. Computing Dataset Probabilities
3. Decoding
4. Decoding Dataset
5. Sampling

Why sample?

1. Model Interpretation
2. Approximate Quantities
3. Interested in Samples, etc.

Learning Strategies

See Learning Principles.

$$\theta_{MLE} = \frac{n_1}{n}$$

Beta distribution (prior)

$$\Pr(\theta \mid \alpha, \beta) \propto \theta^{\alpha-1}(1-\theta)^{\beta-1}$$

$$\alpha = \beta = 1 \Rightarrow MLE$$

$$\alpha = \beta = 2 \Rightarrow \text{Laplace Smoothing}$$

Laplace Smoothing adds extra imaginary '1' and '0'.

Multivariate Binary Density Estimation

Inference tasks, joint, marginal, conditional prob

1. Computing Joint Probabilities $\Pr(x_1, \ldots, x_d)$
2. Computing Marginal Probabilities $\Pr(x_1)$
3. Computing Conditional Probabilities $\Pr(x_1 \mid x_2)$
4. Decoding Dataset
5. Sampling

Product of Bernoullis assumes variables independent.

Generative Classifiers

Model joint prob, $\Pr(x, y)$

Naive Bayes uses product of Bernoullis for $\Pr(x \mid y)$.

Discriminative Classifiers

Model $\Pr(y \mid x)$

Tabular parameterization - fit parameter for $\Pr(y=1 \mid x)$ for each x.

Logistic Regression

$\Pr(y \mid x)$ using sigmoid function.

MLE is convex optimization, gradient descent.

Sigmoid/tanh

$$S(x) = \frac{1}{1+e^{-x}} \in (0,1)$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \in (-1,1)$$

Fundamental Trade-Off

Simple models don't overfit but may underfit, complicated models vice versa.

Neural Networks

One Layer

$$\tilde{y} = \sum_{c=1}^{k} v_c h(w_c^T x_i + b_c) + \beta$$

Learns features and classifier simultaneously

Universal Approximator if layer grows w n examples

Non-Convex problem, SGD works, implicit regularization

Deep Learning

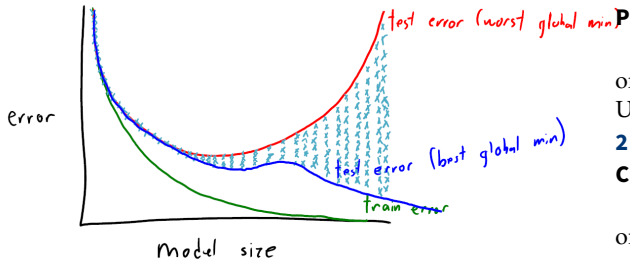
$$\tilde{y} = v^T h(W^4 h(W^3 h(W^2 h(W^1 x))))$$

Vanishing gradient problem - away from origin, gradient nearly 0

ReLU, skip connections, ResNets

NN Heuristics

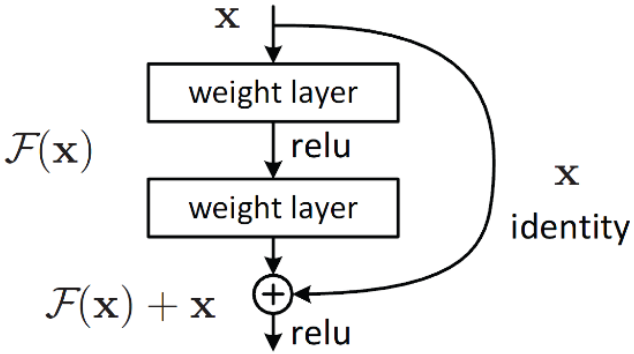
Double Descent Curves



From A2, alternatively increase regularization strength as you add features

ResNets

$$a^{l+2} = h(a^l + W^{l+1} h(W^l a^l))$$



Automatic Differentiation

Code generates gradient code (chain rule)

Advantage - get gradient for same cost as function

Disadvantage - Store all intermediate calculations, same cost as computing function (may have lower cost to calculate gradient for some functions)

CNN

Convolutions transform images, can approximate directional derivatives and integrals at different scales.

Convolution layers, max pooling layers, reduces no. of params, gives spatial invariance

Autoencoders

Output is input Encodes input into bottleneck layer, then decodes back to input

Bottleneck forces compression, cannot just memorize

Non-linear dimensionality reduction

Denoising Autoencoders learn to enhance images

Multi-Label Classification

0-k correct labels

Encoding-Decoding approach: classes shares hidden layers, reduce params, model dependencies

Pixel Labeling

Fully Convolutional networks maintain spatial information at all layers

Upsampling for pixel Classification

2 Part 2

Categorical Density Estimation

Proportion of times different categories appear, ordering of variables in CDF do not matter

$$\Pr(x \mid \Theta) = \theta_1^{I[x=1]} \ldots \theta_k^{I[x=k]}, \sum_c \theta_c = 1$$

θ_C parameterization and unnormalized prob $\tilde{\theta}_C$

$$\theta_{c,MLE} = \frac{n_c}{n}$$

Dirichlet Distribution

Prob Dsb over discrete prob dsbs, used as prior for Categorical

$$\Pr(\Theta \mid \alpha) \propto \prod_c \theta_c^{\alpha_c - 1}$$

MAP w Dirichlet generalizes Laplace smoothing

Categorical Distribution Inference

1. Union, $\Pr(x = 2 \cup x = 3 \mid \Theta)$ or CDF
2. Decoding, $\operatorname{argmax}_c \theta_c$
3. Dataset Prob, $\Pr(X \mid \Theta)$
4. Sampling

Monte Carlo

Approx expectations of random fns.
Generate samples, ave fn value applied to each sample.
Use Indicator function to approx prob of event

Conjugate Priors

Given likelihood, prior leads to posterior of same family

$$x \sim D(\theta), \theta \sim P(\lambda) \Rightarrow \theta \mid x \sim P(\lambda')$$

Likelihood	Prior
Bernoulli/Binomial	Beta
Poisson/Exponential	Gamma
Categorical/Multinomial	Dirichlet
Normal (known var)	Normal
Normal (known mean)	Inverse Gamma
Multivariate Normal (known mean)	Inverse-Wishart

Conditional Independence Assumptions

Hidden data-generating process $\mathcal{D}$

1. IID data
$$x^i \perp x^j \mid \mathcal{D}$$
2. Independence of data given parameters
$$x^i \perp x^j \mid \theta, \mathcal{D}$$
3. Data independent of hyper-params given param
$$X \perp \alpha, \beta \mid \theta$$
4. Discriminative models assume params ind. features
$$w \perp X$$

Bayesian Probabilities

1. Prior ("regularizer")
$$\Pr(\theta \mid \alpha)$$
2. Likelihood
$$\Pr(X \mid \theta)$$
3. Posterior
$$\Pr(\theta \mid X, \alpha)$$
4. Marginal Likelihood
$$\Pr(X \mid \alpha)$$
5. Posterior Predictive
$$\Pr(\tilde{X} \mid X, \alpha)$$

Learning Principles

1. MLE - Maximize prob of data given params (likelihood)
$$\hat{\theta} \in \operatorname*{argmax}_\theta \{\Pr(X \mid \theta)\}$$
$$\hat{x} \in \operatorname*{argmax}_x \{\Pr(x \mid \theta)\}$$
2. MAP - Maximize prob of paras given data (posterior)
$$\hat{\theta} \in \operatorname*{argmax}_\theta \{\Pr(\theta \mid X, \alpha)\}$$
$$\hat{x} \in \operatorname*{argmax}_x \{\Pr(x \mid \theta)\}$$
3. Bayesian ("no learning")
$$\hat{x} \in \operatorname*{argmax}_x \{\Pr(x \mid X, \alpha)\}$$
$$\equiv \operatorname*{argmax}_x \{\int \Pr(\theta \mid X, \alpha) \Pr(x \mid \theta) d\theta\}$$
4. Empirical Bayes - Type II MLE
$$\hat{\alpha} \in \operatorname*{argmax}_\alpha \{\Pr(X \mid \alpha)\}$$
$$\hat{x} \in \operatorname*{argmax}_x \{\Pr(x \mid X, \hat{\alpha})\}$$

Bayesian Learning

Sum/integrate over all params, combine likelihood w prior
Posterior predictive, possibly cost fn for Bayesian decision theory
Strong protection against overfitting - regularizing and averaging

Empirical Bayes

Optimize hyper-params using marginal likelihood
Can optimize large no. of hyper-params w/o validation set
Type II MLE / Evidence Maximization

Hierarchical Bayes

Prior on prior, to model non-IID grouped data

Multi-Class Classification

- Supervised learning w categorical labels
- Tabular Probabilities - exp no. of params
- Bayesian Decision Theory - given cost fn, optimize expected cost under posterior predictive
- Categorical generalization of sigmoid is softmax
  Softmax fn converts k real numbers into probability
$$f_c(Z) = \frac{\exp z_c}{\sum_{c'} \exp z_{c'}}$$
- Multi-Class Neural Networks: Softmax on last layer

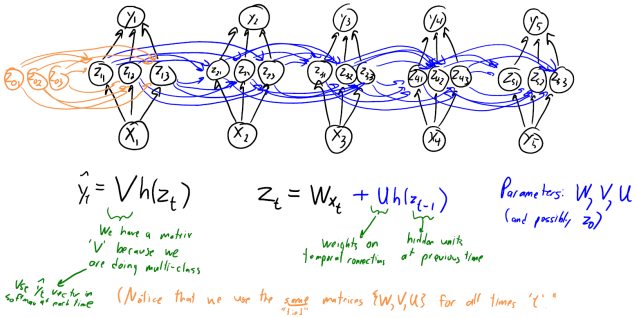
What have we learned?

Layers doing something sensible
ML models easily fooled

- Unpredictable performance on 'out of distribution' images
- Adversarial examples lead to incorrect predictions
- Do not understand context and obvious confounding factors

Harmful biases

Recurrent Neural Networks (RNNs)



k classes for each $\tilde{y}_t$, m hidden units at each time, T times
Total cost: $O(tmd + tkm + tk^2)$
Residual connections to transfer info forward in time more easily
Tied params to model sequences of diff length

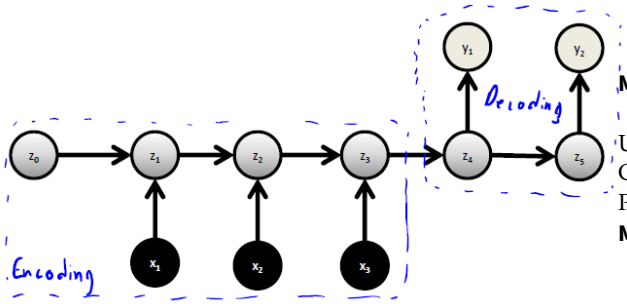
RNN Issues

vanishing/exploding grad

Consider $z^L = U^L z_0$, which diverges exponentially, or converges to zero.

Exponential Forgetting - use skip connections or memory

Sequence-to-Sequence RNNs



Sequence-to-sequence handles output sequences of unknown lengths - BOS at end of input, EOS at end of output.

Multi-modal learning considers input/output of diff formats

Long Short Term Memory (LSTM)

Exponential Forgetting from standard RNNs

Memory cells with Gates

$$a_t = o_t \circ h_o(c_t) \in (-1, 1)$$
$$c_t = f_t \circ c_{t-1} + i_t \circ g_t \in \mathbb{R}$$
$$o_t = h(W_o x_t + U_0 a_{t-1}) \in (0, 1)$$
$$f_t = h(W_f x_t + U_f a_{t-1}) \in (0, 1)$$
$$i_t = h(W_i x_t + U_i a_{t-1}) \in (0, 1)$$
$$g_t = h_o(W_g x_t + U_g a_{t-1}) \in (-1, 1)$$

$\circ$: element-wise multiplication of vectors

Usually gate function h = sigmoid, value function h_0 = tanh.

Model longer-range dependencies than standard RNNs

Multi-Modal Learning - e.g. CNN as encoder and RNN as decoder for image-to-text

Attention and Transformers

Problem w sequence-sequence RNN - all encoding info must be summarized by last state in encoder

Decoder access info from all encoding steps

- Attention: Allow decoder to look at previous states
- Context Vectors: Combine previous states into a fixed-length vector, as opposed to residual connections which requires same length inputs
- Dilated Convolutions for sequences ("with holes")
- Transformer Networks: F̈ully-connectedättention, layers of s̈elf-attentionẗo build context

3 Part 3

Gaussian Density Estimation

Modeling continuous variable samples

Assume symmetric, no outliers, no truncation, uni-modal

Mean and variance parameterization

Inverse of CDF to Uniform samples

$$\Pr(X \mid \mu, \sigma^2) = \frac{1}{(\sigma\sqrt{2\pi})^n} \exp -\frac{\sum_i^n (x^i - \mu)^2}{2\sigma^2}$$

MLE and MAP

MLE for mean and variance is that of the samples

Unless x^i are equal, so NLL not bounded

Conjugate Prior is Gaussian

Posterior Predictive is Gaussian

Multivariate Gaussians

Generalize Univariate Gaussian

$$\Pr(x^i \mid \mu, \Sigma) = \frac{1}{(2\pi)^{\frac{d}{2}} |\Sigma|^{\frac{1}{2}}} \exp -\frac{1}{2}(x^i - \mu)^T \Sigma^{-1} (x^i - \mu)$$

Mean vector μ and positive-definite covariance matrix Σ

Positive definite = all eigenvalues are positive

Prod of ind. Gaussians == diagonal Σ

Models correl. btw. pairs of variables w non-zero off-diagonal

Inference Multivariate Gaussians

Affine transformations are Gaussian

$$x \sim \mathcal{N}(\mu, \Sigma), z = Ax + b \Rightarrow z \sim \mathcal{N}(A\mu + b, A\Sigma A^T)$$

Can go high to low dimension like in PCA

Marginals and conditionals are Gaussian

Can transform samples from ind. Gaussians to multivariate Gaussians

Conditional Independence

Precision Matrix $\Theta = \Sigma^{-1}$

Conditional Independence by off-diagonals of Θ

$\Theta_{ij} \neq 0 \Leftrightarrow$ edge exists in graph

Variables cond. ind. if all paths blocked by cond. variable

Learning Gaussians

MLE: μ = mean of data, Σ = Cov. of data

Posterior and posterior predictive under Gaussian prior on mean is Gaussian

Graphical Lasso uses L1-regularization of Precision Matrix

Sparse Graph represents Conditional Independence

Supervised Learning

Generative classifier is Gaussian Discriminant Analysis (GDA)

Assume $x^i \mid y^i$ is Gaussian

$$\Pr(x^i, y^i = c) = \pi_c \Pr(x^i \mid \mu_c, \Sigma_c)$$

Common $\Sigma = \Sigma_c$ is LDA

If class proportions π_c are equal, class label determined by nearest mean, like ï-nearest neighbour j̈-means clustering.

L2-regularized least squares obtained using Gaussian likelihood and prior

Bayesian Regression

Bayesian Linear Regression - Gaussian posterior

$$y^i \sim \mathcal{N}(w^T x^i, \sigma^2), w_j \sim \mathcal{N}(0, \lambda^{-1})$$

Then the posterior $w \mid X, y$ is a Gaussian centered at w_{MAP}.

Posterior predictive $\tilde{y} \mid X, y, \tilde{x}$ is also Gaussian ("regression w error bars") centered at $w_{MAP}^T \tilde{x}$

Empirical Bayes

Optimize marginal likelihood $\Pr(y \mid X, \sigma^2, \lambda)$.

- Automatic relevance determination (ARD): Can also have separate λ_j, σ^i and optimize
- Bayesian Occam's razor: encourage sparsity and simplicity, prefers simpler models that fit data well
- Bayesian Logistic Regression: Gaussian prior not conjugate, need Approx

Rejection Sampling

Features

1. Ability to evaluate unnormalized $\tilde{p}(x)$.
2. Easy to sample from q.
3. Upper bound M on $\tilde{p}/q$.

Algorithm

1. Sample x from $q(x)$.
2. Sample u from $\mathcal{U}(0, 1)$.
3. Keep sample if $u \leq \frac{\tilde{p}(x)}{Mq(x)}$.

Exact samples from complicated dsb

rejects too many samples in high dimensions

Importance Sampling

Sample from q but reweight by p/q to compute expectation $\mathbb{E}_p[f(x)]$.

Reweights samples from wrong dsb

High variance in high dimensions

Monte Carlo methods

Approx p w empirical dsb over samples

$$p(x) \approx \frac{1}{n} \sum_i \mathcal{I}[x^i = x]$$

Turns inference into sampling

Laplace Approximation

Variational methods approximate p w simpler dsb q

$$p(x) \approx q(x)$$

Turns inference into optimization
Laplace approx - Gaussian centered at MAP aggrees w first 2 derivatives of NLL

End-to-End Learning

Neural Networks w continuous output
Typically trained using squared error, corresponding to Gaussian likelihood
End-to-end: Neural network do all steps, each step in ṗipelineäs differentiable operator

4 Part 4

Exponential Family

$$p(x \mid \theta) = \frac{h(x)\exp(\eta(\theta)^T s(x))}{Z(\theta)}$$

1. $s(x)$ is sufficient statistics - everything relevant to θ about data x
2. η parameter function - controls how params θ interact with statistics. Canonical form $\eta(\theta) = \theta$.
3. Z normalizing - constant ensures sum to 1
4. h support function - contains terms that do not depend on θ

Exponential Families in canonical form are guaranteed to have conjugate priors.

Markov Chains

- Dependencies btw adjacent features, Chain rule of probability
- Markov Assumption: Dsb assuming independence of past given last time

$$\Pr(x_j \mid x_{j-1}, \ldots, x_1) = \Pr(x_j \mid x_{j-1})$$

- Parameterization: Initial probabilities and transition probabilities

$$\Pr(x_1, \ldots, x_d) = \Pr(x_1) \prod_{j=2}^{d} \Pr(x_j \mid x_{j-1})$$

- Homogeneous Markov chains assume same transition probabilities
- Tabular Parameterization

$$\Pr(x_j \mid x_{j-1}) = \theta_{c,c'}$$

$$\theta_{c,c',MLE} = \frac{n_{c,c'}}{n_{k,c'}}$$

- Inhomogenous have different $\Pr(x_j \mid x_{j-1})$ for each j.

Inference in Markov Chains

1. Ancestral Sampling: Sample each variable given previous variables in ordering
2. CK equations: give marginals recursively, exact univariate marginals
3. Stationary dsb π: Marginals as time goes to infinity

$$\pi(c) = \sum_{c'} \Pr(x_j = c \mid x_{j-1} = c')\pi(c')$$

4. Viterbi Decoding: Special case of dynamic programming
5. Forward backward: Computation of all conditionals with two "passes", $O(dk^2)$. Potential representation.

Chapman-Kolgomorov (CK) equations

$$\Pr(x_j) = \sum_{x_{j-1}=1}^{k} \Pr(x_j \mid x_{j-1}) \Pr(x_{j-1})$$

$$\pi_c^j = \Pr(x_j = c)$$

$$T_{cc'}^j = \Pr(x_j = c \mid x_{j-1} = c')$$

$O(dk^2)$ up to time d.

Viterbi Decoding

$$M_j(x_j) = \max_{x_{j-1}} \Pr(x_j \mid x_{j-1}) M_{j-1}(x_{j-1})$$

1. Set $M_1(x_1) = \Pr(x_1), \forall x_1$.
2. Compute $M_2(x_2), \forall x_2$, store value of x_1 leading to best value of each x_2. ...
3. Maximize $M_d(x_d)$ to find value of x_d in decoding.
4. Backtrack to find $x_{d-1}, \ldots, x_1$.

$O(dk^2)$ to search all k^d paths.

Forward Backward

$$\Pr(x_3 \mid x_6) \propto M_6(x_6) \propto V_1(x_1)$$

$M_j(x_j)$ summarizes everything needed up to time j for this x_j value.
V_j summarizes everything needed after time j for this x_j value.
$O(dk^2)$ to compute.

Potential Representation

$$\Pr(x_1, x_2, \ldots, x_d) \propto \left(\prod_{j=1}^{n} \phi_j(x_j)\right)\left(\prod_{j=2}^{d} \psi_j(x_j, x_{j-1})\right)$$

Markov Chain Monte Carlo (MCMC)

Target dsb p as stationary dsb
Samples from Markov chain within Monte Carlo method, burn in and/or thinning

MCMC Issues

1. Burn in - Throw away initial samples when haven't converged to stationary
2. Thinning - only keep every k samples since highly correlated

MCMC Common Implementation

1. Sample from huge number of Markov chains, use final states. Great for parallelization, worry about burn in.
2. Sample from one Markov chain for a long time, use states across time. Worry about thinning.

Metropolis-Hastings

Accepting proposals or keeping same samples $\hat{x}^t$ if

$$u \leq \frac{\tilde{p}(\hat{x}^t)q(x^t \mid \hat{x}^t)}{\tilde{p}(x^t)q(\hat{x}^t \mid x^t)}$$

using proposal dsb $q(\hat{x} \mid x)$.
Note that symmetric q can be ignored in the above ratio.

Gibbs Sampling

Gibbs sampling: Special case of Metropolis-Hastings, sampling one variable at a time given all others.

1. Choose variable j uniformly at Random
2. Update x_j by sampling from conditional

$$x_j \sim \Pr(x_j \mid x_{-j})$$

DAG

Joint prob

$$\Pr(x_1, x_2, \ldots, x_d) = \prod_{j=1}^{d} \Pr(x_j \mid x_{\text{pa}(j)})$$

Independence of prev variables given parent variables

$$\Pr(x_j \mid x_{1:j-1}) = \Pr(x_j \mid x_{\text{pa}(j)})$$

Visualize models/Assumptions

D-Separation

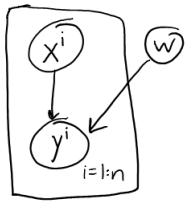
Conditional Independences tested using d-separation - v-structures
A and B are d-separated if all undirected paths P from A to B are "blocked"because at least one of the following holds somewhere on the path

1. P includes a chain with an observed middle node
2. P includes a fork with an observed parent node
3. P includes a v-structure where child and all descendants are unobserved

Lack of d-separation does not imply dependence.

Learning Graphical Models

Independence Assumptions as DAGs, plate notation



Training DAGs decomposes into d supervised learning problems
Parameter learning - fit each $\Pr(x_j \mid \text{pa}(x_j))$ independently

UGMs

Dsb as product of non-negative potentials over subsets of variables

$$\Pr(x_1, x_2, \ldots, x_d) \propto \prod_c \phi_c(x_c)$$

Common choice is pairwise UGM

$$\Pr(x_1, x_2, \ldots, x_d) \propto \prod_{j=1}^{d} \phi_j(x_j) \prod_{(i,j)\in\varepsilon} \psi_{i,j}(x_i, x_j)$$

Ising Model

$$\phi_j(x_j) = \exp(w_j x_j), \phi_{ij}(x_i, x_j) = \exp(v_{ij} x_i x_j)$$

Log-linear models use exp(linear) potentials
Often write log-linear UGMs in exp family form

$$\Pr(x \mid w) = \frac{\exp(w^T F(x))}{Z(w)}$$

Learning Log-Linear UGMs

Convex NLL trained with gradient descent, but gradient requires inference, normalizing constant

$$\nabla_w - \log \Pr(x \mid w) = -F(x) + \mathbb{E}_{x|w}[F(x)]$$

Approx training methods include pseudolikelihood (objective fn) and variational methods
pseudolikelihood turns learning into d single-variables problem

$$\Pr(x_1, \ldots, x_d) \approx \prod_{j=1}^{d} \Pr(x_j \mid x_{\text{nei}(j)})$$

Younes Algorithm

Younes algo for MCMC within SGD

1. Run Gibbs sampling for a short time starting from x_{k-1} with current w^k. e.g. 1 pass to generate x^k
2. SGD Update:

$$w^{k+1} = w^k + \alpha_k(F(x) - F(x^k))$$

Inference Tasks

1. Compute $\Pr(x)$ for an assignment to variables x.
2. Generate sample x.
3. Compute univariate marginals $\Pr(x_j)$.
4. Compute decoding $\text{argmax}_x \Pr(x)$.
5. Compute univariate conditional $\Pr(x_j \mid x_{j'})$.

Conditional Random Fields

$$\Pr(y \mid x, w) = \frac{1}{Z(x, w)} \exp(w^T F(x, y))$$

Generalize logistic regression
Add features to UGM

$$\Pr(y_1, \ldots, y_k \mid x_1, \ldots, x_k) \propto \exp(\sum_{c=1}^{k} y_c w^T x_c + \sum_{(c,c')\in E} y_c y_{c'} v)$$

Multi-label model explicitly models label dependencies
Deep structured models learn features in UGMs

Inference in Graphical Models

Markov chain inference methods extend to trees for DAGs and UGMs
General Graphs: Inference can be hard in DAGs/UGMs
Uncond. sampling and learning easy in DAGs
Exact inference in discrete DAGs easy for trees - exponential in ẗreewidthöf graph

Need UGMs in NNs?

Factor 1: Data Size

1. Small dataset: Helpful to have direct dependencies
2. Large dataset: Hidden layers should reflect

Factor 2: Evaluate Model

- If measure "how many pixels right", UGMs may not help.
- If measure "get whole sentence right", UGMs may help.

5 Part 5

Mixture Models

$$\Pr(x \mid \pi, \Theta) = \sum_{c=1}^{k} \pi_c \Pr(x \mid \Theta_c)$$

Latent-variable representation of mixtures with cluster variables z^i
Ancestral sampling
Responsibility = prob example belongs to cluster

Mixture of Gaussians

Prob as convex comb. of Gaussian densities

$$\Pr(x \mid \pi, \mu, \Sigma) = \sum_{c=1}^{k} \pi_c \Pr(x \mid \mu_c, \Sigma_c)$$

Model arbitrary continuous indentities (üniversal approximator")

Mixture of Bernoullis

Mixture of Bernoullis model dependencies btw discrete variables

$$\Pr(x \mid \pi, \Theta) = \sum_{c=1}^{k} \pi_c \prod \theta_{cj}$$

Soft-clustering of examples

Notation

1. $$\pi \in [0,1]^k, \sum_i \pi_i = 1$$

   π_c is prior prob that example is in cluster c.
   Parameter learned from data.

2. $$R \in [0,1]^{n\times k}, \sum_c r_c^i = 1$$

   Responsibility r_c^i is posterior prob that example i is in cluster c.

Computing values is inference task (assumes known params)

3. $$z \in \{1, 2, \ldots, k\}^n$$

Category z^i is actual mixture generating example i.
Nuissance parameter - unknown variable not a parameter

Imputation

Alternate btw finding best value of latent variables, and updating params

1. Find most likely cluster z^i for each example i

$$z^i \in \underset{c}{\operatorname{argmax}} \Pr(z^i = c \mid x^i, \Theta)$$

2. Update mixture probs as proportion of examples cluster is responsible for

$$\pi_c = \frac{1}{n} \sum_{i=1}^{n} I[z^i = c]$$

3. Update prod of Bernoullis based on examples cluster is responsible for

$$\theta_{cj} = \frac{1}{n_c} \sum_{i=1}^{n} I[z^i = c] x_j^i$$

Expectation Maximization (EM)

EM : Algorithm for optimization w hidden variables
Instead of imputation, s̈oftässignments to hidden variables
Maximizes log likelihood, weighted by imputations of hidden variables
Local optimum
General algorithm for parameter learning with missing data

EM for Prod of Bernoullis

1. Find responsibility of cluster z^i for each example i

$$r_c^i = \Pr(z^i = c \mid x^i, \Theta)$$

2. Update mixture probs as proportion of examples cluster is responsible for

$$\pi_c = \frac{1}{n} \sum_{i=1}^{n} r_c^i$$

3. Update prod of Bernoullis based on examples cluster is responsible for

$$\theta_{cj} = \frac{1}{\sum_{i=1}^{n} r_c^i} \sum_{i=1}^{n} r_c^i x_j^i$$

General EM

1. E-Step: Compute responsibilities $r_c^i, \forall i, c$ for current Θ^t.
2. M-Step: Optimize weighted complete-data log-likelihood

$$\Theta^{t+1} \in \underset{\Theta}{\operatorname{argmax}} \sum_{i=1}^{n} \sum_{z^i=1}^{k} r_c^i \log \Pr(x^i, z^i \mid \Theta)$$

EM Properties

1. Useful when complete-data log-likelihood easy to optimize
2. Monotonically increases likelihood $\Pr(X \mid \Theta^{t+1}) \geq \Pr(X \mid \Theta^t)$
3. Does not need step size
4. Tends to satisfy constraints automatically
5. Iterations are parameterization independent
6. Notorious for converging to bad local optima (to be expected for hard problems)
7. Converges to stationary point under weak assumptions
8. At least as fast as gradient descent with constant step size
9. Converges faster as entropy of hidden variables decreases
10. Arbitrarily faster than gradient descent in "best case"

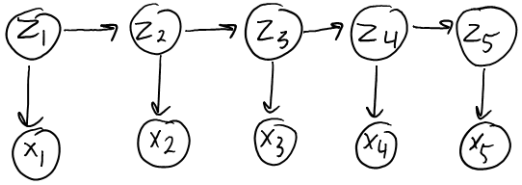
Kernel Density Estimation (KDE)

KDE : Non-parametric density method

$$\Pr(x^i) = \frac{1}{n} \sum_{j=1}^{n} \Pr(x^i \mid x^j, \sigma^2 I)$$

Center a mixture on each datapoint
Data visualization and low-dimensional density estimation
Basis of mean-shift clustering

Hidden Markov Models (HMMs)



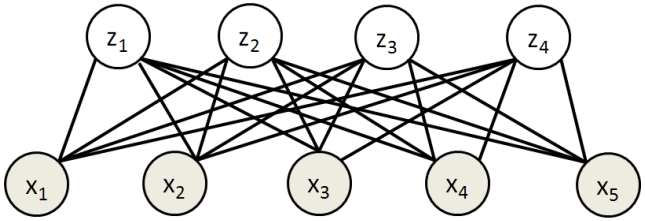
$$\Pr(x_1, \ldots, x_d, z_1, \ldots, z_d) = \Pr(z_1) \prod_{j=2}^{d} \Pr(z_j \mid z_{j-1}) \prod_{j=1}^{d} \Pr(x_j \mid z_j)$$

HMMs model time-series with hidden per-time cluster

Restricted Boltzmann Machines (RBMs)

RBMs : UGMs with binary hidden variables

$$\Pr(x, z) = \frac{1}{Z} \prod_{j=1}^{d} \phi_j(x_j) \prod_{c=1}^{k} \phi_c(z_c) \prod_{j=1}^{d} \prod_{c=1}^{k} \phi_{jc}(x_j, z_c)$$



Normally with log-linear parameterization

$$\Pr(x, z) \propto \exp\left(\sum_{j=1}^{d} w_j x_j + \sum_{c=1}^{k} v_c z_c + \sum_{j=1}^{d} \sum_{c=1}^{k} w_{jc} x_j z_c \right)$$

Pairwise edges only btw visible and hidden
Efficient block Gibbs sampling for inference/learning
Deep Boltzmann machines stack RBMs into deep density estimation model